

TEMA 9-B: CARACTERÍSTICAS DEL TEXTO

Como lo mencionamos en videos anteriores, la extracción de características es un paso importante para cualquier problema de análisis de datos. No importa qué tan buena sea la estrategia de análisis o el modelo que utilicemos, si introducimos características deficientes, vamos a obtener malos resultados.

Acá vamos a centrarnos en la pregunta: ¿cómo hacemos para realizar ingeniería de características para datos de texto? En otras palabras, ¿cómo transformamos un texto determinado en forma numérica para que pueda incorporarse a los modelos de procesamiento de lenguaje natural o aprendizaje automático?

La representación de características es un paso común en cualquier proyecto de ciencia de datos, ya sea que los datos sean texto, imágenes, videos o voz. Sin embargo, la representación de características del texto suele ser mucho más complicada en comparación con otros formatos de datos.

Para entender esto, veamos algunos ejemplos de cómo se pueden representar numéricamente otros formatos de datos. Primero, consideremos el caso de las imágenes. Digamos que queremos crear un clasificador que pueda distinguir imágenes de gatos de imágenes de perros. Ahora, para entrenar un modelo de aprendizaje automático para realizar esta tarea, necesitamos alimentarlo con imágenes (etiquetadas). ¿Cómo alimentamos imágenes a un modelo aprendizaje automático?

La forma en que se almacena una imagen en una computadora es en forma de una matriz de píxeles donde cada celda $[i, j]$ de la matriz representa el píxel i, j de la imagen. El valor real almacenado en la celda $[i, j]$ representa la intensidad del píxel correspondiente en la imagen. Esta representación matricial representa con precisión la imagen completa. El vídeo es similar: un vídeo es sólo una colección de fotogramas donde cada fotograma es una imagen. Por tanto, cualquier vídeo puede representarse como una colección secuencial de matrices, una por cuadro, en el mismo orden.

Ahora, ¿Qué pasa con el texto? Resulta que representar texto no es sencillo, de ahí que le vamos a dedicar un video completo centrado en varios esquemas para abordar esta cuestión. Nos dan un fragmento de texto y nos piden que encontremos una forma para representarlo matemáticamente. En general, este proceso va a involucrar primero una tarea de preprocesamiento y limpieza para luego seleccionar una estrategia para convertir el texto en una representación numérica.

LIMPIEZA DE CARACTERÍSTICAS

Una de las primeras tareas de limpieza que podemos realizar en un texto es la eliminación de stopwords o palabras de parada. Las stopwords son palabras que son muy comunes y que generalmente no aportan información significativa para el análisis del texto. Estas palabras comunes, incluyen términos como "y", "el", "la", "de", "en", "por", "a", "para", entre otros.

El propósito de la eliminación de las stopwords es variado. En primer lugar, podemos mencionar la reducción de la dimensionalidad. Al eliminar las stopwords, se reduce la cantidad de palabras únicas en el texto, lo cual simplifica el análisis y mejora la eficiencia de los algoritmos de aprendizaje automático.

Por otro lado, se busca mejorar la relevancia. Las stopwords suelen aportar poco valor informativo y pueden introducir ruido en los datos. Al eliminarlas, se mejora la relevancia de las palabras restantes que tienen más significado en el contexto del análisis. Por último, permite la optimización de recursos. Al trabajar con menos palabras, se requiere menos espacio de almacenamiento y menos poder de procesamiento, lo cual es beneficioso para tareas que involucran grandes volúmenes de datos.

¿Cómo es el proceso de eliminación de stopwords? El paquete NLTK de Python, contiene una lista de palabras vacías definidas por lingüistas para muchos idiomas. También podemos encontrar varias

listas de palabras vacías en la web. Sólo es necesario importar el paquete, seleccionar el listado de las stopwords en el idioma correspondiente y realizar la eliminación.

Hay otras formas, más estadísticas, de llegar al concepto de "palabras comunes". Las estadísticas de frecuencia son excelentes para filtrar palabras comunes específicas del corpus, así como palabras vacías de propósito general. Por ejemplo, la frase "New York Times" y cada una de las palabras individuales que contiene aparecen con frecuencia en el conjunto de datos del diario New York Times. De manera similar, la palabra "boletín" aparece a menudo en la frase "Boletín Oficial" en el corpus constituido por los boletines oficiales de la nación publicados diariamente. Estas palabras tienen significado en general, pero no dentro de esos corpus en particular. Una lista típica de palabras vacías captará las palabras vacías generales, pero no las específicas del corpus.

Observar las palabras más frecuentes puede revelar problemas de análisis y resaltar palabras normalmente útiles que aparecen demasiadas veces en el corpus.

En la práctica, resulta útil combinar el filtrado basado en frecuencia con una lista de palabras irrelevantes. También está la difícil cuestión de dónde colocar el límite. Lamentablemente no existe una respuesta universal. La mayoría de las veces, el límite debe determinarse manualmente y es posible que sea necesario volver a examinarlo cuando cambie el conjunto de datos.

Dependiendo de la tarea, es posible que también sea necesario filtrar palabras raras. Estas pueden ser palabras verdaderamente oscuras o errores ortográficos de palabras comunes. Para un modelo estadístico, una palabra que aparece sólo en uno o dos documentos es más ruido que información útil. Por ejemplo, supongamos que la tarea es categorizar negocios según sus reseñas en Tripadvisor y una sola reseña contiene la palabra "galimatías". ¿Cómo se podría saber, basándose en esta única palabra, si el negocio es un restaurante, un hotel o un bar?

Las palabras raras no sólo son poco fiables como predictores, sino que también generan una sobrecarga computacional. En muchos corpus de documentos más del 60% del vocabulario aparece raramente. Esta es la llamada distribución de cola pesada y es muy común en los datos del mundo real. El tiempo de entrenamiento de muchos modelos estadísticos de aprendizaje automático aumenta linealmente con la cantidad de características, y las palabras raras conllevan un gran costo de cálculo y almacenamiento sin mucha ganancia adicional.

Las palabras raras se pueden identificar y recortar fácilmente según las estadísticas de recuento de palabras. Alternativamente, sus recuentos se pueden agregar en un único contenedor especial, que puede servir como característica adicional. Las palabras habituales conservan sus propios recuentos, que pueden filtrarse aún más mediante listas de palabras vacías u otros métodos basados en la frecuencia. Las palabras raras pierden su identidad y se agrupan en un único contenedor.

Un problema común al momento de preprocesar un texto es que diferentes variaciones de la misma palabra se cuentan como palabras separadas. Por ejemplo, "flor" y "flores" son tokens o unidades del lenguaje técnicamente diferentes, al igual que "nadador" y "nadar", aunque tienen un significado muy parecido. Sería bueno si todas estas variaciones diferentes se asignaran a la misma palabra.

La **lematización** es un proceso lingüístico que consiste en, dada una forma flexionada (es decir, en plural, en femenino, conjugada, etc), hallar el lema correspondiente. El lema es la forma que por convenio se acepta como representante de todas las formas flexionadas de una misma palabra. Es decir, el lema de una palabra es la palabra que nos encontraríamos como entrada en un diccionario tradicional: singular para sustantivos, masculino singular para adjetivos, infinitivo para verbos. Por ejemplo, decir es el lema de dije, pero también de diré o diremos. Este proceso tiene en cuenta la estructura morfológica de las palabras y su contexto gramatical en la oración. Utiliza diccionarios y reglas gramaticales para determinar la forma base correcta. En la figura de la transparencia vemos un ejemplo de la aplicación de la lematización de la librería Spacy.

Otro proceso similar es el **stemming**. El stemming, o derivación, es otro proceso de normalización de texto utilizado en el procesamiento de lenguaje natural (NLP) y aprendizaje automático, similar a la lematización. Sin embargo, a diferencia de la lematización, el stemming corta los sufijos de las palabras para reducirlas a su raíz o “estema”. Esta raíz no necesariamente es una palabra real en el idioma, pero ayuda a reducir la variabilidad de las formas de las palabras.

La mayoría de las herramientas de derivación se centran en el idioma inglés, aunque se están realizando esfuerzos bastantes decentes para otros idiomas. En la figura de la transparencia vemos un ejemplo de cómo ejecutar un stemmer a través del paquete NLTK de Python. Como pueden ver, lo aplicamos sobre las mismas tres palabras que usamos en la lematización, y el resultado no es tan bueno.

La derivación tiene un costo de cálculo. Que el beneficio final supere el costo depende de la aplicación. También vale la pena señalar que la derivación podría perjudicar más que ayudar ya que dos palabras que no están relacionadas pueden quedar reducidas a la misma forma.

Cuando preprocesamos un texto realizaremos una lematización o una derivación, pero no ambas. La decisión de qué proceso usar dependerá de la tarea posterior que realizaremos con el texto.

Mencionamos el concepto de token o unidades del lenguaje, palabras de parada, palabras raras, etc. Pero, ¿cómo sabe una computadora qué es una palabra? Un documento de texto se representa digitalmente como una cadena, que es básicamente una secuencia de caracteres. También se puede encontrar texto semiestructurado en forma de una página HTML por ejemplo. Pero incluso con las etiquetas y la estructura agregadas, la unidad básica sigue siendo una cadena. ¿Cómo se convierte una cadena en una secuencia de palabras? Esto implica tareas de parseo y tokenización, que vamos a analizar a continuación.

- La **tokenización** es el proceso de dividir un texto en unidades más pequeñas llamadas "tokens". Los tokens pueden ser palabras, frases, oraciones o incluso caracteres individuales, dependiendo de la granularidad requerida para la tarea específica.
- El **parseo** es el proceso de analizar una cadena de texto para extraer su estructura gramatical. Implica descomponer el texto en componentes sintácticos y determinar sus relaciones y funciones dentro de una oración.
- Finalmente, el etiquetado de partes del discurso (**POS tagging**, por sus siglas en inglés) es un proceso en el procesamiento de lenguaje natural que implica asignar etiquetas gramaticales a cada palabra en una oración, de acuerdo con su categoría gramatical. Estas categorías incluyen sustantivos, verbos, adjetivos, adverbios, preposiciones, conjunciones, entre otros.

Para todos estos procesos las librerías más usadas en procesamiento del lenguaje natural nos ofrecen funciones.

BOLSA DE PALABRAS

Ya sea que estemos construyendo características o modelos de aprendizaje automático, es bueno que el resultado sea simple e interpretable. Las cosas simples son fáciles de probar y las características y modelos interpretables son más fáciles de depurar que los complejos. Obviamente, las características simples e interpretables no siempre conducen al modelo más preciso, pero es una buena idea comenzar de manera simple y solo agregar complejidad cuando sea absolutamente necesario.

Para datos de texto, podemos comenzar con una lista de estadísticas de recuento de palabras llamada bolsa de palabras. Una lista de recuentos de palabras no hace ningún esfuerzo especial para encontrar entidades interesantes, como nombres o entidades. Para tareas sencillas como por ejemplo clasificar un documento, las estadísticas de recuento de palabras suelen ser suficientes. Esta técnica también se puede utilizar en la recuperación de información, donde el objetivo es recuperar el conjunto de documentos que son relevantes para una consulta de texto de entrada. Ambas tareas se realizan bien

con funciones a nivel de palabra porque la presencia o ausencia de ciertas palabras es un excelente indicador del contenido del tema del documento.

En la generación de características a través de bolsa de palabras (BoW), un documento de texto se convierte en un vector de recuentos. El vector contiene una entrada para cada palabra posible en el vocabulario. Si la palabra (por ejemplo, "oso ") aparece tres veces en el documento, entonces el vector de características cuenta con 3 en la posición correspondiente a esa palabra. Si una palabra del vocabulario no aparece en el documento, se cuenta como 0.

La bolsa de palabras convierte un documento de texto en un vector plano. Es "plano" porque no contiene ninguna de las estructuras del texto. El texto original es una secuencia de palabras. Pero una bolsa de palabras no tiene secuencia; simplemente recuerda cuántas veces aparece cada palabra en el texto. Por lo tanto, el orden de las palabras en el vector no es importante y no es conservado.

La bolsa de palabras tampoco representa ningún concepto de jerarquía de palabras. Por ejemplo, el concepto de "animal" incluye "perro", "gato", "cuervo", etc. Pero en una representación de bolsa de palabras, todas estas palabras son elementos iguales del vector.

Lo importante aquí es la geometría de los datos en el espacio de características. En un vector de bolsa de palabras, cada palabra se convierte en una dimensión del vector. Si hay n palabras en el vocabulario, entonces un documento se convierte en un punto en un espacio de n dimensiones. Es difícil visualizar la geometría de algo más allá de dos o tres dimensiones, por lo que vamos a tener que usar nuestra imaginación. La Figura en la transparencia muestra cómo se ve nuestra oración de ejemplo en el espacio de características bidimensional correspondiente a las palabras "cachorro" y "lindo" y en la otra figura vemos tres oraciones en un espacio 3D correspondiente a las palabras "cachorro", "extremadamente" y "lindo".

La bolsa de palabras no es perfecta. Dividir una oración en palabras individuales puede destruir el significado semántico. Por ejemplo, "no está mal" significa semánticamente "decente" o incluso "bueno" pero "no" y "mal" son palabras que expresan por separado más un sentimiento negativo. "Perro de juguete" y "juguete para perros" son cosas muy diferentes y el significado se pierde si tomamos las palabras sueltas "juguete" y "perro". Es fácil encontrar muchos ejemplos de este tipo. Los n -gramas, que vamos a analizar a continuación, alivian parte del problema, pero no son una solución definitiva. Es bueno tener en cuenta que la bolsa de palabras es una heurística simple y útil, pero está lejos de ser una comprensión semántica correcta del texto.

N-GRAMAS

La bolsa de n -gramas, es una extensión natural de bolsa de palabras. Un n -grama es una secuencia de n tokens. Una palabra es esencialmente 1 token, también conocido como unigrama. Después de la tokenización, el mecanismo de conteo puede recopilar tokens individuales en recuentos de palabras o contar secuencias superpuestas como n -gramas.

Los n -gramas conservan más de la estructura de secuencia original del texto y, por lo tanto, la representación de la bolsa de n -gramas puede ser más informativa. Sin embargo, esto tiene un costo. Teóricamente, con k palabras únicas, podría haber k^2 bigramas únicos. En la práctica, no hay tantas, porque no todas las palabras pueden seguir a otras. Sin embargo, normalmente hay muchos más n -gramas distintos ($n > 1$) que palabras. Esto significa que la bolsa de n -gramas es un espacio de características mucho más grande y disperso (¿recuerdan que cuando hablamos de la dimensionalidad mencionamos los problemas de los conjuntos de datos con muchas dimensiones y dispersos?). También significa que los n -gramas son más caros de calcular, almacenar y modelar. Cuanto mayor es n , más rica es la información y mayor el costo.

TF-IDF

Una representación de una bolsa de palabras es sencilla de generar, pero está lejos de ser perfecta. Si contamos todas las palabras por igual, algunas terminan siendo enfatizadas más de lo necesario.

Tf-idf es una simple variación del enfoque de la bolsa de palabras. Significa frecuencia de términos/ frecuencia de documentos inversa. En lugar de observar los recuentos sin procesar de cada palabra en cada documento en un conjunto de datos, tf-idf analiza un recuento normalizado donde cada recuento de palabras se divide por la cantidad de documentos en los que aparece esta palabra. Es decir, tomamos en cuenta el número de veces que aparece la palabra w en el documento d y lo multiplicamos por el cociente de la cantidad total de documentos (N) y el número de documentos en los que aparece la palabra w . Esta fracción es lo que se conoce como frecuencia inversa de documentos. Si una palabra aparece en muchos documentos, entonces su frecuencia inversa de documento es cercana a 1. Si una palabra aparece en solo unos pocos documentos, entonces la frecuencia inversa de documento es mucho mayor.

Además, la fórmula incluye una transformación logarítmica de la frecuencia inversa del documento. El logaritmo convierte 1 en 0 y hace que los números grandes (aquellos mucho mayores que 1) sean más pequeños. Entonces, una palabra que aparece en cada documento se eliminará efectivamente, y una palabra que aparece en muy pocos documentos tendrá un recuento aún mayor que antes.

Veamos un ejemplo sencillo, tenemos 3 documentos. Sería deseable que el número que nos da la representatividad de cada palabra sea menor para la palabra “es” que aparece en todos los documentos y sea mayor para “cachorro”. Usando la fórmula que vimos tenemos que para la palabra cachorro la frecuencia es 0,48 y para la palabra “es” es cero.

WORD EMBEDDINGS

Existen otras formas un poco más ricas a la hora de representar numéricamente un texto y las veremos en el apunte siguiente.